Nature@Cloud

Cloud Computing and Virtualization

André Bento - 106930
Pedro Loureiro - 107059
João Santos - 107365

Abstract

This report describes the full Nature@Cloud implementation. Javassist-based worker instrumentation, DynamoDB-backed metrics persistence, periodic linear regression retraining, complexity-aware load balancing, and `CPU + complexity` driven autoscaling. We detail the design and decision making for each component, justified by experiments comparing five candidate profiling metrics. The final design selects `weighted_instructions_External_Invocations` as the complexity metric, routes requests to the least in-flight-complexity EC2 worker (falling back to Lambda on overflow or when workers are temporarily unreachable), and scales the worker pool using a mixed load signal with drain-aware termination.

1 Architecture Overview

The system runs entirely within AWS and consists of five cooperating components. **EC2 VM Workers** execute the three workloads (Fractals, GrayScott, DNA) inside a Java web server instrumented with a Javassist agent that accumulates per-request complexity metrics. The **Load Balancer (LB)**, a `t3.micro` instance with an Elastic IP inside a dedicated VPC, is the sole entry point: it receives every request, estimates the request's complexity, selects a target worker, and forwards it, or invokes a Lambda if no worker can absorb the load. The **Auto Scaler (AS)**, co-located with the LB, periodically evaluates a combined load signal and adds or removes worker instances via the AWS SDK. The same instance also hosts the **Retraining Service**, which runs every 5 minutes and, when significant new data is available, retrains the regression models and stores the updated coefficients in the MSS. The **Metrics Storage System (MSS)** uses two DynamoDB tables: `hhs-workload-metrics` (per-request complexity records written by workers) and `hhs-regression-models` (trained model coefficients). **Lambda Workers** provide a fast, stateless overflow tier for burst traffic; each workload type has a dedicated Lambda function.

2 Instrumentation

2.1 Metric Selection

We evaluated six candidate Javassist metrics, and all comparisons were performed in a single-threaded, unloaded environment so that wall-clock time served as a proximity for the computational cost.

MethodCalls had the lowest overhead (≈80%) but correlated poorly with complexity, the call count does not reflect the amount of computation inside loops.

BasicBlocks improved correlation at the cost of higher overhead (≈139%), but still missed dense computational loops where a single block executes a lot of instructions.

LoopIterations offered good information gain with comparatively low overhead for our workloads, because most complexity is loop-bound. However, loop semantics differ significantly across workload types, making it impractical for general use.

BytecodeInstructions, the default metric used in the lab, provided substantially better correlation. However, its overhead (≈170%) was high relative to the improvement over BasicBlocks, since every instruction requires an atomic increment at runtime.

WeightedInstructions The overhead (≈196%) is the high, but the information gain justifies it. The key insight is that instruction weights are assigned *at instrumentation time* per basic block, each block receives a pre-computed cost scalar derived from the instruction mix (memory accesses, arithmetic, branches). At runtime, only one atomic addition per block is needed (the same cost as BasicBlocks) but the accumulated value reflects execution weight rather than a flat count. This design recovers most of the signal quality of raw BytecodeInstructions at BasicBlocks-level per-block overhead, yielding the best overhead/signal tradeoff for our use case, but it still has limitations for library calls with variable cost.

The DNA algorithm's execution time lives almost entirely inside Java standard-library calls: `String.substring()` and `String.equals()` that are invoked on every iteration of the inner loop. The base WeightedInstructions agent only instruments classes in the `pt.ulisboa.tecnico.cnv.*` package, the bodies of `java.lang.String` methods are never transformed. What the counter sees is a single `INVOKEVIRTUAL` opcode, charged a fixed weight of 6 regardless of how much work the invocation performs. The real cost of `String.equals()` is proportional to the length of the strings being compared.

WeightedInstructionsExternalInvocations extends `WeightedInstructions` and is scoped to the `pt.ulisboa.tecnico.cnv.dna` package. At instrumentation time, Javassist intercepts every `String.substring` and `String.equals` call and injects a counter increment proportional to the number of characters processed, the substring length or the receiver string's length, respectively.
At runtime this is just two extra operations per call: a length read and an atomic counter increment, both accumulated in the same per-thread counter as the static block weights.

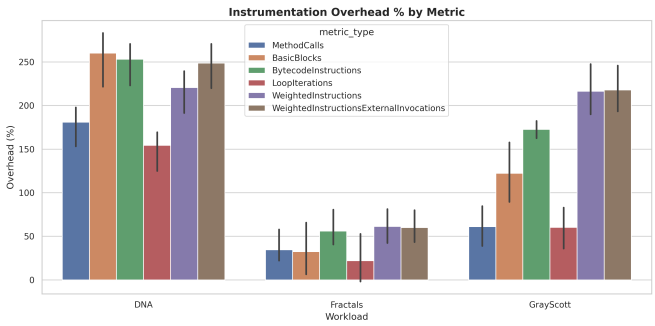


Figure 1: Instrumentation overhead (%) by metric and workload.

2.2 Metric-to-Time Correlation

To explicitly compare instrumentation signals against wall-clock runtime, we computed Pearson correlation r between each metric and `execution_time_ms` on the profiling CSVs.

Metric	DNA	Fractals	GrayScott	Avg. $\lvert r \rvert$
MethodCalls	0.047	n/a	0.997	0.522
BasicBlocks	0.339	0.982	0.998	0.773
BytecodeInstructions	0.212	0.992	0.999	0.734
LoopIterations	0.068	0.984	0.800	0.617
WeightedInstructions	0.293	0.949	0.982	0.741
WeightedInstructions + ExtInvoc.	**0.999**	**0.995**	**0.999**	**0.998**

Table 1: Metric-vs-time correlation (r). MethodCalls/Fractals is undefined because sampled values are nearly constant (no variance). WeightedInstructions + ExtInvoc. extends WeightedInstructions with runtime character-count tracking for DNA string operations.

For complexity-vs-time Weighted Instructions with External Invocations correlates strongly with runtime across all workloads, with an average $\lvert r \rvert$ of 0.998. The improvement over the base WeightedInstructions metric (average $\lvert r \rvert = 0.741$) is dramatic for DNA (from 0.293 to 0.999), while Fractals and GrayScott also show marginal gains (from 0.949 to 0.995, and from 0.982 to 0.999, respectively). Since the external invocation probe is scoped exclusively to DNA classes, these small improvements are likely attributable to natural measurement variance in the test environment rather than any change in instrumentation.

2.3 Threshold Calibration

To determine the per-worker complexity budget used by the LB overflow policy, we ran a sweep of requests of increasing size on an isolated `t3.micro` and recorded the weighted-instruction count at which CPU utilization reached saturation. This value becomes the *complexity threshold*.

3 Data Structures

3.1 External Storage (DynamoDB)

Two DynamoDB tables form the persistent layer: `hhsworkloadmetrics` stores per-request records with a composite key of instance ID (pk) and `epochMs#requestId` (sk), carrying the workload name, request timestamp, weighted-instruction complexity, and the input parameter map; `hhsregressionmodels` stores trained model coefficients with the workload name as pk and the training timestamp as sk, where the `model` field is a JSON-serialised `RegressionModel` or `DualRegressionModel`. Both tables are written by workers (metrics) and the retraining service (models); the LB reads only from the model table via `ModelCatalog`.

3.2 In-Memory Structures

TargetPool maintains a `ConcurrentMap` of live workers. Each `BackendTarget` carries an `AtomicLong inflightComplexity`, an `AtomicInteger inflightRequests`, and an `AtomicBoolean retiring`, all updated lock-free. A `ConcurrentLinkedQueue` of `WaiterEntry` objects forms the FIFO overflow queue, each entry wraps a `CompletableFuture<Void>` and the queued complexity so the waiting thread blocks on the future while the client connection is held open.
An `AtomicLong totalQueuedComplexity` mirrors the queue depth for the AS `/metrics` endpoint without traversing the queue.
ModelCatalog holds one model reference per workload, refreshed every five minutes from DynamoDB, avoiding per-request remote reads.
AutoScaler state includes a `Map<String, Integer>` of per-instance consecutive health-failure counts, a `ConcurrentHashMap.newKeySet()` of draining instance IDs, an `Instant` last-scale-out timestamp for cooldown enforcement, and a daemon `CachedThreadPool` for running drain tasks off the main polling loop.

4 Complexity Prediction and Model Design

4.1 Regression Models

We trained linear regression models using the workload input parameters as features. For DNA, complexity grows non-linearly with sequence lengths, which violates the linearity assumption of standard regression. We address this by training in *log-space*, both the complexity target and the sequence-length features are log-transformed before fitting. At prediction time the balancer applies the same transforms and exponentiates the result to recover the original scale.

Table 2 reports training R^2.

Model	R^2	Samples
DNA plain (linear)	0.4723	132
DNA (stopOnFirst=false)	0.9581	66
DNA (stopOnFirst=true)	0.8569	66
Fractals	0.8140	47
GrayScott	0.7418	64

Table 2: Model training R^2. Partitioning DNA by `stopOnFirst` and switching to log-space regression raises the average R^2 from 0.47 to >0.90.

The dramatic improvement of the DNA dual-model (R^2 from 0.47 to >0.90 on average) was achieved by combining two techniques: partitioning on `stopOnFirst`, and log-space regression, which linearizes the underlying power-law relationship. At request time, the LB selects the model branch

based on the `stopOnFirst` parameter and applies the log transformation before computing the complexity estimate.

4.2 ModelCatalog and Retraining

The `ModelCatalog` component caches model coefficients in memory and refreshes them from `hhs-regression-model` every 5 minutes, avoiding per-request DynamoDB reads. `ModelTrainingService` runs on a 5-minute timer, fetches new metric records from `hhs-workload-metrics` using a cursor (last-seen timestamp), and rewrites updated coefficients only when a minimum sample count is reached, preventing underfitted models from entering production. To avoid the model drifting towards stale historical data, training always uses the 500 most recent records rather than the full table history, keeping the model responsive to current workload behaviour. If the catalog has no valid model for a request, the LB falls back to workload-specific parameter heuristics so that routing is never blocked.

5 Load Balancer

5.1 Infrastructure

The LB runs as a Java web server on a dedicated `t3.micro` instance with an Elastic IP, placed inside a VPC. It is the sole public-facing endpoint; worker instances have no public IPs. The LB instance is assumed to be always available (no replication required per specification).

5.2 Worker Discovery and Health Checking

`TargetPool` maintains a live set of worker instances. Periodically, it queries the AWS EC2 API for instances carrying a project-specific tag, then probes each candidate's `/health` endpoint. Instances that fail the health check are removed from the pool; newly tagged instances are added. This decouples the LB from the AS: the scaler only needs to launch or terminate instances with the correct tag, and the LB will discover them automatically within the next polling cycle.

5.3 Routing Algorithm

When a request arrives, the LB executes the following policy:

1. **Estimate complexity.** `ModelCatalog` returns a predicted weighted-instruction count $\hat{c}$ for the request parameters. If $\hat{c} > \Theta$, the request exceeds the capacity of a single worker even in isolation; it is rejected immediately with HTTP 503. Lambda is not offered in this case because the workload would also exceed Lambda's execution budget.
2. **Select target and retry loop.** `TargetPool` picks the healthy instance with the lowest current in-flight complexity L_i (ties broken by in-flight count). The LB attempts up to five different workers in increasing-load order, retrying on both transport errors and HTTP 5xx responses.
3. **Overflow check per candidate.** For each candidate, if $L_i + \hat{c} > \Theta$:
   - If $\hat{c} \leq \Theta/2$: forward to Lambda. Because workers are tried in least-load order, if the best worker cannot absorb the request then no worker can, so Lambda is the right fallback. Small-to-medium requests are cheap on Lambda and start instantly.
   - If $\hat{c} > \Theta/2$: skip Lambda. Large requests are disproportionately expensive on Lambda; instead, continue trying other workers. If all workers are exhausted, the request is queued until a worker frees capacity. This bounds Lambda cost under sustained heavy load.
4. **No-worker Lambda fallback.** If all retry attempts are exhausted because no healthy workers are reachable (e.g. during an auto-scaler replacement cycle, or the pool is empty), and $\hat{c} \leq \Theta/2$, the LB attempts Lambda before resorting to the wait queue. The wait queue is triggered on either condition: capacity is fully blocked (`capacityBlocked`), or the pool has no live targets at all (`noTargets`). This avoids an unnecessary queuing delay for requests that fit within Lambda's budget when workers are temporarily unavailable. If Lambda also fails, the request is enqueued and retried once a worker becomes available.
5. **Reserve and forward.** $\hat{c}$ is added to L_i before the request is dispatched. When the worker responds, $\hat{c}$ is subtracted, releasing capacity for future requests. The backend connection carries a **10-minute timeout**: this value was chosen because the largest request that completed successfully in profiling ran for approximately 8–9 minutes. A timeout set below that would abort legitimate long-running jobs.

Fault-tolerance mechanisms applied during forwarding, transport retry, pessimistic load accounting, Lambda failover, and the FIFO wait queue are detailed in Section 7.

6 Auto Scaler

6.1 Load Signal

The AS computes a combined load metric every polling interval:

$$\text{systemLoad} = \alpha \cdot \overline{\text{CPU}} + \beta \cdot \overline{\text{Cplx}}$$

where $\overline{\text{CPU}}$ is the average CPU utilization across active workers (obtained from CloudWatch, expressed as a percentage 0–100) and $\overline{\text{Cplx}}$ is the average per-worker in-flight complexity fetched from the LB `/metrics` endpoint, normalized to the same 0–100 scale relative to Θ. Using both signals prevents a failure mode where many lightweight requests keep CPU low but complexity saturated, or vice versa. We set $\alpha = 0.3$ and $\beta = 0.7$, weighting complexity more heavily because CPU lags behind actual load on our instrumented JVM workloads.

6.2 Complexity Normalization

$\overline{\text{Cplx}}$ is computed as

$$\overline{\text{Cplx}} = \frac{\text{totalInflightComplexity} + \text{queuedComplexity}}{\text{workerCount} \cdot \Theta}$$

where Θ is the `MAX_COMPLEXITY_PER_WORKER` constant (the same per-worker capacity ceiling used by the LB overflow check). Including `queuedComplexity` (requests waiting for a free worker slot) ensures that demand beyond current worker capacity naturally drives $\overline{\text{Cplx}}$ above 100%, pushing $\beta \cdot \overline{\text{Cplx}}$ into scale-out territory without any special-case logic. Using the same Θ in both the LB and the AS keeps the two components calibrated to the same per-worker capacity, so a complexity level that triggers LB overflow also drives the AS toward scale-out.

6.3 Scale-Out and Scale-In Policy

Scale-out uses a two-tier response based on load severity. If `systemLoad` > 160 (twice the upper threshold), the AS launches *two* EC2 workers simultaneously from the pre-baked AMI; this aggressive response absorbs burst load faster and reduces the time windows where the LB must spill to Lambda. If `systemLoad` > 80 (but ≤ 160), one worker is launched. In both cases a scale-out timestamp is recorded to enforce a cooldown window that prevents an immediate scale-in from undoing the action. Both scale-out paths operate on `effectiveInstances`, which excludes workers currently mid-drain, so a drain does not falsely suppress further scale-out.

If `systemLoad` < 20 and the cooldown since the last scale-out has elapsed, the AS selects the least-loaded worker for retirement and initiates a graceful drain: the LB is signalled to stop assigning new work to that instance, and the AS polls in a background thread until in-flight requests reach zero (or a timeout expires) before terminating the EC2 instance. The full drain protocol is described in Section 7.6.

6.4 Fault Detection

During each polling cycle the AS probes every worker's `/test` endpoint. Two consecutive `ConnectException` failures are required before the instance is terminated; a single transient failure is not acted upon. The full detection and replacement logic is described in Section 7.5.

7 Fault Tolerance

No error is returned to the client unless every recovery path has been exhausted. The mechanisms below operate at several independent layers, so a failure at one layer is absorbed by the next before the client is aware.

7.1 Transport-Level Retry

When the LB forwards a request it selects up to five different backend workers in ascending in-flight-complexity order, using a `triedIds` set to prevent re-selecting the same worker. Both transport failures (connection refused, `IOException`, or connect timeout) and HTTP 5xx responses trigger a retry on the next candidate. The `HttpClient` carries a 10-second connect timeout and a 600-second per-request timeout, sized to cover the longest observed workload (roughly 8–9 minutes) without aborting legitimate long-running jobs.

7.2 Pessimistic Load Accounting

Before forwarding, `reserveLoad` atomically increments the target's `inflightComplexity`.
`releaseLoad` is called in a `finally` block, so the reservation is undone even if an exception propagates mid-dispatch. This prevents two concurrent routing decisions from both fitting into the same worker slot and keeps accounting consistent across all failure paths, including retries.

7.3 Lambda Failover

When a selected worker is over capacity ($L_i + \hat{c} > \Theta$) and the request fits Lambda's budget ($\hat{c} \leq \Theta/2$), the LB invokes the corresponding Lambda function with up to two internal retries. If the pool is entirely empty, for example during an AS replacement cycle, Lambda is tried before the wait queue for eligible requests, avoiding unnecessary queuing.
Lambda is reserved for requests with $\hat{c} \leq \Theta/2$, larger requests always stay on EC2, keeping serverless cost bounded even under sustained load.

7.4 FIFO Wait Queue

Requests that exhaust all EC2 retries and the Lambda fallback are enqueued in a Concurrent Linked Queue, each entry pairs a `CompletableFuture<Void>` with the request's predicted complexity. The dispatching thread blocks on the future, holding the client HTTP connection open so no error is visible to the caller.
Entries are dequeued in FIFO order on two events: (a) an in-flight request completes (`releaseLoad` releases the head entry immediately), or (b) target discovery adds new healthy workers (`signalWaiters` releases one entry per new backend). Queued requests are thus served as soon as capacity appears, without polling or busy-waiting.

7.5 Worker Failure Detection

The AS probes every worker's `/test` endpoint each polling cycle with a 3-second timeout. Crucially, the check inspects the full exception chain and only increments the failure counter on a `ConnectException`, which indicates the JVM is not listening. Other exceptions (read timeouts, DNS errors) are

treated as transient and do not count as failures. Two consecutive `ConnectException` failures are required before the worker is terminated (`UNHEALTHY_TERMINATION_THRESHOLD = 2`), the counter resets to zero on any successful probe. No replacement is launched immediately after termination, instead, the next polling cycle detects that the worker count has dropped below the minimum and launches a new instance through the normal scale-out path.

7.6 Graceful Scale-In Drain

When the AS decides to retire a worker it first signals the LB via `POST /retire?ip=...`. The LB sets an Atomic Boolean retiring flag on the `BackendTarget`. `chooseTarget` skips retiring instances so no new requests are assigned. The AS then polls `/retire-status?ip=...` every 5 s in a dedicated daemon thread from a Cached Thread Pool, leaving the main polling loop free to run its next evaluation cycle concurrently. Once the inflight count reaches zero, or a 300-second drain timeout expires, the EC2 instance is terminated. The draining instance is tracked in `drainingInstanceIds` and excluded from `effectiveInstances`, so an ongoing drain never suppresses a concurrent scale-out decision. If the LB does not acknowledge the `/retire` signal, the AS terminates the instance immediately rather than blocking indefinitely.

7.7 Pool Discovery Resilience

If the AWS EC2 API call that discovers workers returns an error, the LB keeps its current target set unchanged rather than clearing it. This prevents routing blackouts caused by transient IAM or API failures. On the other hand, newly discovered instances must pass a `/test` health check before being admitted to the pool, ensuring that a freshly booted worker does not receive requests until its JVM is ready to serve them.